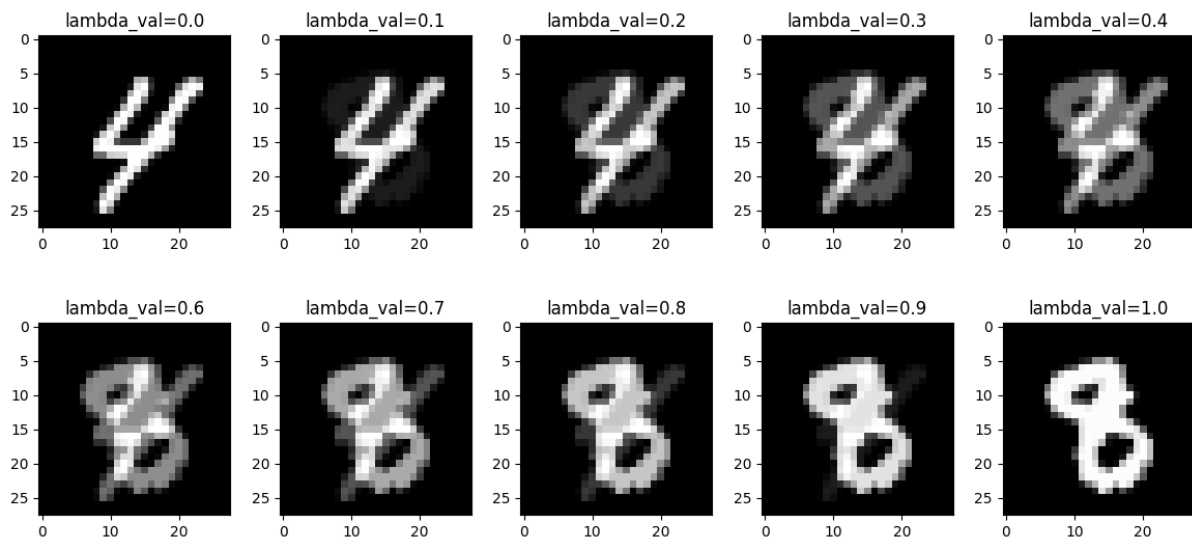


HW3 report

학번 비공개 심현성

1. Interpolation of 4 and 8 in the image space.



Code for answer 1.

```
# Answer 1
img4 = digits[4][5]
img8 = digits[8][5]

lambda_range=np.linspace(0,1,10)

fig, axs = plt.subplots(2, 5, figsize=(15, 6))
fig.subplots_adjust(hspace = .5, wspace=.001)
axs = axs.ravel()

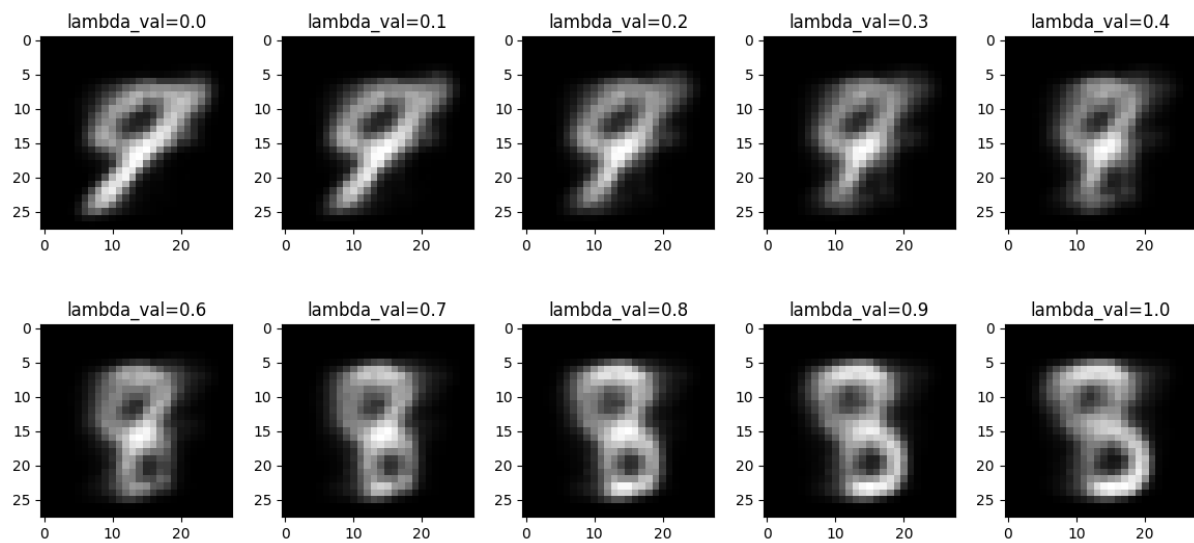
for ind, l in enumerate(lambda_range):
    inter_image = (1 - l) * img4 + l * img8

    inter_image = to_img(inter_image)

    image = inter_image.numpy()

    axs[ind].imshow(image[0,0,:,:], cmap='gray')
    axs[ind].set_title('lambda_val='+str(round(l,1)))
plt.show()
```

2. Interpolation of decoded 4 and 8 in the latent space.



Code for answer 2.

```
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed()
```

(I defined a new function "set_seed" for reproducibility in the beginning part of the skeleton codes.)

```
# Answer 2
vae.eval()
lambda_range=np.linspace(0,1,10)

fig, axs = plt.subplots(2, 5, figsize=(15, 6))
fig.subplots_adjust(hspace = .5, wspace=.001)
axs = axs.ravel()

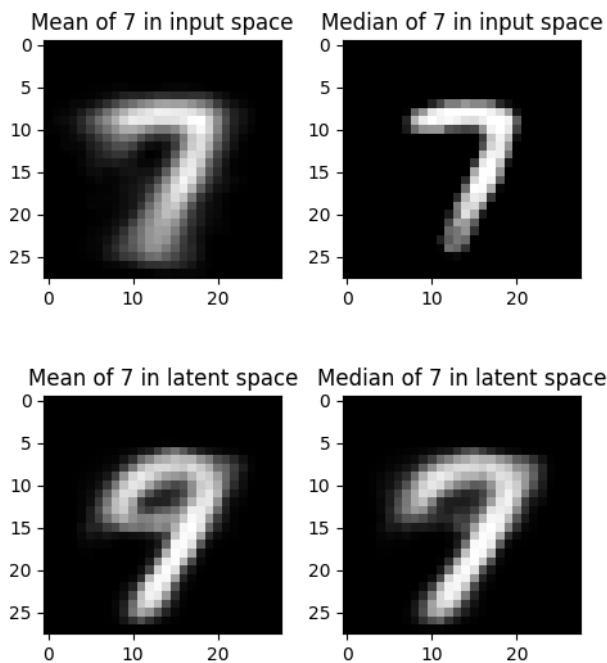
for ind, l in enumerate(lambda_range):
    inter_image=interpolation(1-float(l), vae, img4, img8)

    inter_image = to_img(inter_image)

    image = inter_image.numpy()

    axs[ind].imshow(image[0,0,:,:], cmap='gray')
    axs[ind].set_title('lambda_val='+str(round(l,1)))
plt.show()
```

3. Mean and median of 7 in the input and latent space.



According to the figure on the left, in the input space, the mean image appears blurry because it is the result of pixel-wise averaging, while the median image looks sharper because it is the result of taking median value of each pixel.

In the latent space, the decoded image from the mean latent vectors looks weird because there might be some outliers (incorrectly encoded latent vectors of 7) and it is the result of averaging all latent vectors. However, the median one produces cleaner reconstruction because

it is less influenced by outliers.

Code for answer 3.

```
# Answer 3
vae.eval()

imgs7 = torch.cat(digits[7], dim=0).to(device)

mean_x = imgs7.mean(dim=0)
median_x = imgs7.median(dim=0).values

latents = []
with torch.no_grad():
    for img7 in digits[7]:
        img7 = img7.to(device)
        latent_mu, latent_logvar = vae.encoder(img7)
        latent = vae.latent_sample(latent_mu, latent_logvar)
        latents.append(latent.squeeze(0))
latents = torch.stack(latents)

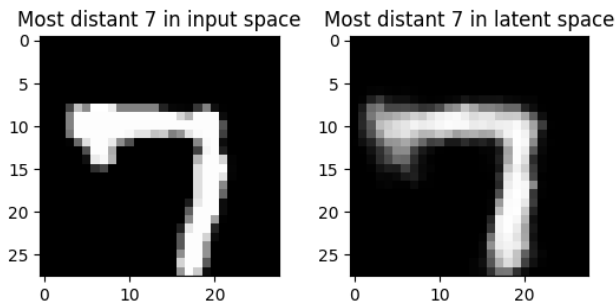
mean_z = latents.mean(dim=0)
median_z = latents.median(dim=0).values

decoded_mean_z = vae.decoder(mean_z.unsqueeze(0))
decoded_median_z = vae.decoder(median_z.unsqueeze(0))

fig, axs = plt.subplots(2, 2, figsize=(6, 6))
fig.subplots_adjust(hspace=.5, wspace=.001)
axs = axs.ravel()

axs[0].imshow(mean_x.squeeze().cpu().numpy(), cmap='gray')
axs[0].set_title('Mean of 7 in input space')
axs[1].imshow(median_x.squeeze().cpu().numpy(), cmap='gray')
axs[1].set_title('Median of 7 in input space')
axs[2].imshow(decoded_mean_z.squeeze().cpu().detach().numpy(), cmap='gray')
axs[2].set_title('Mean of 7 in latent space')
axs[3].imshow(decoded_median_z.squeeze().cpu().detach().numpy(), cmap='gray')
axs[3].set_title('Median of 7 in latent space')
plt.show()
```

4. The strangest (farthest from the mean) sample of 7 in the input and latent spaces.



They look similar.

Code for answer 4.

```
# Answer 4
vae.eval()

diff_x = imgs7 - mean_x
dists_x = torch.norm(diff_x.view(diff_x.size(0), -1), dim=1)
farthest_x = imgs7[dists_x.argmax()]

dists_z = torch.norm(latents - mean_z, dim=1)
farthest_z = latents[dists_z.argmax()]
decoded_farthest_z = vae.decoder(farthest_z.unsqueeze(0))

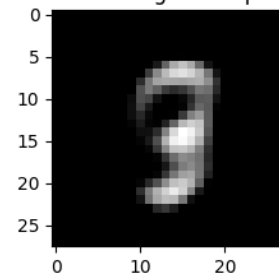
fig, axs = plt.subplots(1, 2, figsize=(6, 6))
fig.subplots_adjust(hspace = .5, wspace=.3)
axs = axs.ravel()

axs[0].imshow(farthest_x.squeeze().cpu().numpy(), cmap='gray')
axs[0].set_title('Most distant 7 in input space')
axs[1].imshow(decoded_farthest_z.squeeze().cpu().detach().numpy(), cmap='gray')
axs[1].set_title('Most distant 7 in latent space')
plt.show()
```

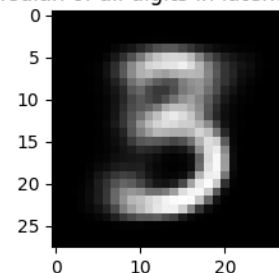
5. Median of all digits in the input and latent spaces.

According to the figure on the right, they look like digit 3 in both spaces. However, the decoded image from the latent space appears to be a clearer digit because VAE tries to reconstruct an image based on learned representations in the latent space, whereas the median of input space is just a simple pixel-wise median value, making it look less structured.

Median of all digits in input space



Median of all digits in latent space



Code for answer 5.

```
# Answer 5
vae.eval()

all_imgs = []
all_latents = []

with torch.no_grad():
    for img_batch, _ in test_dataloader:
        img_batch = img_batch.to(device)
        latent_mu, latent_logvar = vae.encoder(img_batch)
        latent = vae.latent_sample(latent_mu, latent_logvar)
        all_imgs.append(img_batch)
        all_latents.append(latent)

all_imgs = torch.cat(all_imgs, dim=0)
median_x = all_imgs.median(dim=0).values

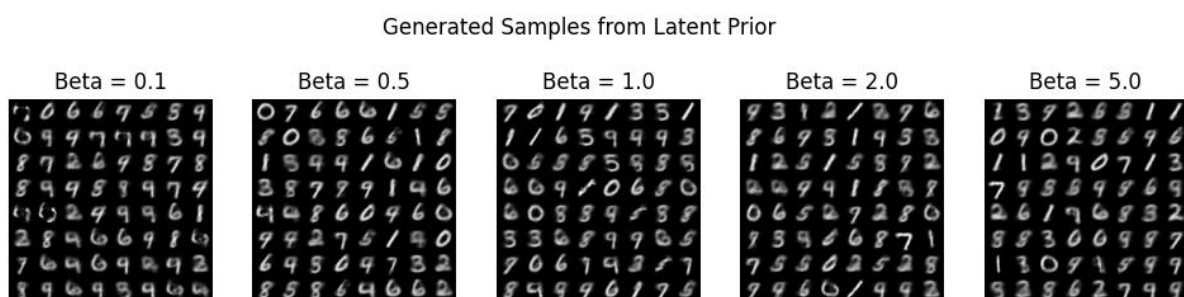
all_latents = torch.cat(all_latents, dim=0)
median_z = all_latents.median(dim=0).values
decoded_median_z = vae.decoder(median_z.unsqueeze(0))

fig, axs = plt.subplots(1, 2, figsize=(6, 6))
fig.subplots_adjust(hspace = .5, wspace=.7)
axs = axs.ravel()

axs[0].imshow(median_x.squeeze().cpu().numpy(), cmap='gray')
axs[0].set_title('Median of all digits in input space')
axs[1].imshow(decoded_median_z.squeeze().cpu().detach().numpy(), cmap='gray')
axs[1].set_title('Median of all digits in latent space')
plt.show()
```

6. Training five models with various strengths of regularization. Discussion of the quality of generated samples and reconstruction errors of seen and unseen data.

I trained five VAE models for 10 epochs with different values of regularization weight, beta, of [0.1, 0.5, 1.0, 2.0, 5.0]. In the figure of "Generated Samples from Latent Prior" below, I sampled some random latent vectors from a standard normal distribution and decoded them using each model. For small beta (e.g., 0.1), there are some crushed and unclear images, and for large beta (e.g., 5.0), almost all images look like certain digits. It means that with higher strength regularization, a model tends to generate higher-quality images.



According to the bar chart below, the reconstruction errors of seen (train) and unseen (test) data increase as beta increases, and the gap between them also increases. This is because a larger beta forces the posterior distribution to align more strictly with the prior, which limits the ability of the model to memorize details of each input image, leading to higher reconstruction errors, especially on training data.

In conclusion, there exists a trade-off between reconstruction accuracy and quality of generated samples.



Code for answer 6.

```

1 # Answer 6
2 train_losses_savepath = 'train_losses.pt'
3 test_losses_savepath = 'test_losses.pt'
4
5 variational_betas = [0.1, 0.5, 1.0, 2.0, 5.0]
6 vaes = []
7 train_losses = []
8
9 def vae_loss_beta(recon_x, x, mu, logvar, beta=1):
10     recon_loss = F.binary_cross_entropy(recon_x.view(-1, 784), x.view(-1, 784), reduction='sum')
11     kldivergence = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
12     return recon_loss + beta * kldivergence
13
14 for beta in variational_betas:
15     vae_savepath = f'vae_2d_10_beta{beta}.pth'
16
17     vae = VariationalAutoencoder().to(device)
18     optimizer = torch.optim.Adam(params=vae.parameters(), lr=learning_rate, weight_decay=1e-5)
19
20     if os.path.isfile(vae_savepath) and os.path.isfile(train_losses_savepath):
21         print(f'Loading pretrained VAE from {vae_savepath}')
22         vae.load_state_dict(torch.load(vae_savepath))
23     else:
24         vae.train()
25
26         train_loss_avg = []
27
28         print(f'Training VAE with beta = {beta}')
29
30         for epoch in range(10):
31             train_loss = 0
32
33             for image_batch, _ in train_dataloader:
34                 image_batch = image_batch.to(device)
35
36                 # vae reconstruction
37                 image_batch_recon, latent_mu, latent_logvar = vae(image_batch)
38
39                 # reconstruction error
40                 loss = vae_loss_beta(image_batch_recon, image_batch, latent_mu, latent_logvar, beta)
41
42                 # backpropagation
43                 optimizer.zero_grad()
44                 loss.backward()
45
46                 # one step of the optimizer (using the gradients from backpropagation)
47                 optimizer.step()
48
49                 train_loss += loss.item()
50
51             train_loss /= len(train_dataloader)
52             print(f'Epoch [xd / xd] average reconstruction error: {x} % (epoch+1, 10, train_loss)')
53
54             train_losses.append(train_loss)
55             torch.save(vae.state_dict(), vae_savepath)
56
57         vaes.append(vae)
58
59 if os.path.isfile(train_losses_savepath):
60     train_losses = torch.load(train_losses_savepath)
61 else:
62     torch.save(train_losses, train_losses_savepath)

```

```

1 test_losses = []
2
3 for vae, beta in zip(vaes, variational_betas):
4     vae.eval()
5
6     if not os.path.isfile(test_losses_savepath):
7         test_loss = 0
8         with torch.no_grad():
9             for image_batch, _ in test_dataloader:
10                 image_batch = image_batch.to(device)
11                 image_batch_recon, latent_mu, latent_logvar = vae(image_batch)
12                 loss = vae_loss_beta(image_batch_recon, image_batch, latent_mu, latent_logvar, beta)
13                 test_loss += loss.item()
14             test_loss /= len(test_dataloader)
15             test_losses.append(test_loss)
16             print(f'Test loss for beta = {beta}: {test_loss}')
17
18 if os.path.isfile(test_losses_savepath):
19     test_losses = torch.load(test_losses_savepath)
20 else:
21     torch.save(test_losses, test_losses_savepath)

```

```

1 x = np.arange(len(variational_betas)) # [0, 1, 2, 3, 4]
2 width = 0.35 # Width of a bar
3
4 plt.figure(figsize=(8, 5))
5 plt.bar(x - width/2, train_losses, width, label='Train Loss', color='skyblue')
6 plt.bar(x + width/2, test_losses, width, label='Test Loss', color='orange')
7
8 plt.xticks(x, [str(b) for b in variational_betas])
9 plt.xlabel('Beta (Regularization Strength)')
10 plt.ylabel('Average Reconstruction Loss')
11 plt.title('Reconstruction Loss vs Beta')
12 plt.legend()
13 plt.grid(axis='y', linestyle='--', alpha=0.7)
14 plt.tight_layout()
15 plt.show()

```

```

1 set_seed()
2
3 sample_images = []
4
5 with torch.no_grad():
6     for vae in vaes:
7         latent = torch.randn(64, latent_dims).to(device)
8         samples = vae.decoder(latent).cpu()
9         sample_images.append(samples)
10
11 fig, axs = plt.subplots(1, 5, figsize=(12, 3))
12 axs = axs.ravel()
13
14 for i, (samples, beta) in enumerate(zip(sample_images, variational_betas)):
15     recon_grid = torchvision.utils.make_grid(samples[:64], nrow=8, padding=1)
16     axs[i].imshow(recon_grid.permute(1, 2, 0).numpy(), cmap='gray')
17     axs[i].set_title(f'Beta = {beta}')
18     axs[i].axis('off')
19
20 plt.suptitle("Generated Samples from Latent Prior")
21 plt.show()

```

7. A method that can generate diverse random samples given a class.

To generate diverse random samples given a class label, we can extend the original VAE model into Conditional VAE (CVAE). During training, CVAE model takes both an image x and its corresponding class label y as input, then it can learn detailed class-specific features of x . The decoder then reconstructs an image $\hat{x}$ from y and the latent vector z . After training, we can fix a class label y (e.g., 8) and sample diverse latent vectors $z \sim \mathcal{N}(0, I)$ to generate diverse images of the specific class.